THE SOLUTION BOOK

SCOPE IT, SCAFFOLD IT

One worked scope — follow along.

One fully worked example, so you can see what a Week 5 capstone scope and MVP skeleton look like — a brief, a stack, a Claude architecture, a tested CLAUDE.md, and a first route live.

YOUR DELIVERABLE HAS FIVE PARTS

- Product brief
- Tech stack
- Claude architecture
- CLAUDE.md
- First route live

WHAT, WHY, WHERE, WHEN & HOW

The whole deliverable in five plain-English lines.

WHAT	Produce a complete capstone brief — problem, user, solution, tech stack with justification, and your Claude architecture (system prompt, MCP tools, Routines). Then scaffold the MVP: a tested **CLAUDE.md**, a folder structure, and a first route live on Replit.
WHY	This is the leap from automating workflows to **shipping products**. A scoped, scaffolded capstone means Week 6 is a build sprint, not a planning scramble — you show up ready to build.
WHERE	**Claude Code** is the engineering brain; **CLAUDE.md** is its context; MCPs wire it to GitHub and Supabase; Lovable / Bolt.new scaffold; **Replit** deploys to a live URL.
WHEN	Week 5 of the Fellowship — about 75–90 minutes. Scope a real product you would actually ship: for your college, your community, or a market you understand.
HOW	Write the brief → choose and justify the stack → design the Claude architecture → write and test CLAUDE.md → scaffold the repo and get a first route live on Replit.

YOUR TOOLBOX — CLAUDE

The Claude power-tools that are the engineering brain of your build.

C Claude Code

TERMINAL · AI IDE

The agentic coding environment — it reads `CLAUDE.md`, edits files, runs commands and opens PRs, with your approval on every change.

M CLAUDE.md

YOUR REPO ROOT

The project context file: goal, stack, conventions and a file map. Claude Code reads it every session — your most important file.

X MCPs & Connectors

CLAUDE CODE CONFIG

Claude's hands on GitHub, Supabase and APIs during the build — read code, read/write data, open PRs. You grant the least access it needs.

P Projects & Skills

SIDEBAR → PROJECTS

Projects hold the full app context across sessions; Skills store reusable component and API-integration patterns you recall on demand.

YOUR TOOLBOX — BUILD & DEPLOY

The editors, scaffolders and platforms that carry your build to a live URL.

U Cursor

AI EDITOR

Tab-complete, Cmd-K inline edit, chat sidebar and composer — four ways to build with AI directly in the editor.

L Lovable · Bolt.new

PROMPT → APP

Describe the app and get a full-stack scaffold — UI plus database — then iterate component by component by prompt.

S Supabase

THE BACKEND

Postgres database, auth, real-time and edge functions — the backend your app and Claude read and write.

R Replit · GitHub

DEPLOY · VERSION

Replit turns localhost into a live URL; GitHub versions your code and can trigger each deploy.

SET UP & CONNECT (EXAMPLES)

1 Set up Claude Code + CLAUDE.md

1. Install Claude Code and open your project folder as the workspace.
2. Create a `CLAUDE.md` at the repo root — goal, stack, conventions, file map.
3. Start a session: Claude reads CLAUDE.md as its project context automatically.
4. Describe what to build → review the diff → accept or refine. You own the code.

Safety — Claude Code shows you the diff before anything is written — review every change before you accept it.

2 Connect an MCP — GitHub & Supabase

1. Add the MCP server for GitHub (and Supabase) in your Claude Code config.
2. Authorise it with a scoped token you paste into the config — never your password.
3. Now Claude can read your repo, open PRs, and read/write your database during the build.
4. Grant the least access it needs; keep write actions behind your approval.

Like this — MCPs are Claude's hands on your real tools — you decide what it can touch.

WRITE & CREATE (EXAMPLES)

Two files that keep your build consistent — a context file and a reusable Skill.

1 Write CLAUDE.md — the context file

1. At the repo root, create `CLAUDE.md` — Claude Code reads it every session.
2. Put the goal, the stack, folder conventions, and a short "never do" list.
3. Keep it short and current — it is the single most important file in the project.
4. Test it: ask Claude "what are we building?" — the answer should match your brief.

Like this — a good CLAUDE.md is a brief a teammate could build from without asking you a thing.

2 Create a Skill — /component

1. Customize → Skills — or in Cowork: "+" → Record a skill.
2. Write a SKILL.md: name, a one-line description (the trigger), and your component rules.
3. Zip the folder (folder as root) → Upload → enable.
4. Run `/component` to generate UI the same way every time.

DESIGN THE PRODUCT

One worked scope — a real product, scoped in four lines.

PROBLEM

Students cram the night before and forget — they don't know which topics are weak.

USER

A final-year student revising for end-sems across five subjects.

SOLUTION

Upload your notes → Claude makes adaptive quizzes, tracks weak topics, and sends a 10-question set each morning.

STACK

Lovable (UI) · Supabase (DB + auth) · Claude API (quiz + grading) · Replit (deploy).

Why it works

A sharp problem, a named user, a one-line solution and a defensible stack — that is a product a stranger understands and you can build in a sprint.

WRITE CLAUDE.MD

CLAUDE.md = Goal · Stack · Conventions · Never. It is Claude Code's memory of your project.

GOAL

An adaptive study app: upload notes → quizzes → weak-topic tracking → a daily set.

STACK

Vite + React (Lovable), Supabase (Postgres + auth), Claude API, Replit deploy.

CONVENTIONS

TypeScript; components in /src/components; one table per entity; keys in .env, never commit.

NEVER

invent facts not in the notes; hardcode keys; ship a route with no loading + error state.

YOUR OUTPUT — A REAL CLAUDE.MD

CLAUDE.MD · REPO ROOT

```
# StudyLoop — CLAUDE.md
GOAL: Adaptive study web app. Upload notes → generate quizzes → track weak topics → daily 10-Q set.
STACK: Vite + React (Lovable) · Supabase (Postgres + auth) · Claude API (quiz gen + grading) · Replit.
CONVENTIONS: TypeScript; UI in /src/components; one Supabase table per entity; env keys in .env (never commit).
CLAUDE'S ROLE: write quiz-generation + grading; return STRICT JSON; cite the note each question came from.
NEVER: invent facts not in the user's notes; hardcode API keys; ship a route without a loading + error state.
```

THE BUILD — PICK ONE

Your capstone direction — you will ship it in Week 6. Scope it now.

- **A · AI Study Assistant** — course-specific knowledge, chat or voice Q&A, and progress tracking.
- **B · AI Startup MVP** — an agentic tool solving a real market problem, pitched as a product.
- **C · AI Automation Service** — a packaged automated system you could sell as a service.
- **D · AI Creative Platform** — a tool others use to create, produce, or publish with AI.

Must include

Claude visible and central (MCPs, Routines, or API calls live), a real user, and a slice you can demo. Tools: Claude (Code, Projects, MCPs, API), Lovable / Bolt.new, Supabase, Replit, GitHub, plus the stack your idea needs.

Submit as — a product brief + a scaffolded repo with CLAUDE.md + a first deployed route on Replit.

NOW SCOPE & SCAFFOLD

Six moves from a blank page to a live first route.

1. Write the brief — problem, user, solution, success criteria, in four short lines.
2. Choose the stack — frontend, backend, deploy, model — with a one-line justification each.
3. Design the Claude architecture — system prompt, the MCP tools it needs, any Routines.
4. Write CLAUDE.md — goal, stack, conventions, never-do — and test it in Claude Code.
5. Scaffold the repo — the folder structure plus a first component or route.
6. Deploy to Replit — a real URL that loads, even if it only shows one screen.

WHAT GOOD LOOKS LIKE

- One crisp problem, user and solution
- A stack you can defend pick by pick
- A CLAUDE.md a teammate could build from

COMMON MISSES

- A scope too big for one sprint
- A stack chosen with no reason
- A vague or out-of-date CLAUDE.md

That's a full Week 5 scope + skeleton. Yours can differ — **grade the thinking, not the match.**